

ASSIGNMENT 2

Email Template Generator - Logic and Working

1. Overview

This project is a Node.js microservice designed to generate professional, context-aware email templates using Generative AI (Google Gemini). It exposes a RESTful API endpoint that accepts user inputs (purpose, recipient, tone) and returns a fully formatted email.

2. Architecture

The application follows a standard **Controller-Service** architecture to ensure separation of concerns:

- **src/app.js**: The entry point that initializes the Express server and routes.
- **src/controllers/emailController.js**: Handles HTTP requests, input validation, and sends responses.
- **src/services/aiService.js**: Encapsulates the business logic for interacting with the Google Gemini AI model.
- **src/config/env.js**: Manages environment variables (API keys, ports).

3. Detailed Logic Flow

Step 1: Server Initialization

- The application starts an Express.js server on the port defined in `.env` (default: 3000).`
- Middleware is configured to parse JSON request bodies (`express.json()`) and handle CORS (`cors()`).
- The route `/api/email-template` is mapped to the createEmailTemplate` controller function.`

Step 2: Request Handling (Controller)

When a `POST` request is made to /api/email-template`:`

1. **De-structuring**: The controller extracts ``purpose`, `recipient_name`, and `tone` from req.body`.`
2. **Validation**: It checks if all three fields are present. If any are missing, it returns a ``400 Bad Request` error.`
3. **Service Call**: If valid, it invokes `aiService.generateEmail()` passing the input data.

4. ****Error Handling****: Catches any errors during the process and returns a `500 Internal Server Error` if the generation fails.

Step 3: AI Generation (Service)

The `generateEmail` function in `src/services/aiService.js` performs the core logic:

1. ****Timing Start****: Records the current timestamp (`Date.now()`).
2. ****Prompt Construction****: Dynamically builds a text prompt for the AI.
 - *Example Prompt*: "You are an expert email copywriter. Write a short, customer-friendly email template... Context: Purpose: [purpose], Recipient: [name], Tone: [tone]..."
3. ****AI Call****: Uses the `@google/generative-ai` library to send the prompt to the `gemini-pro` model.
4. ****Response Processing****: Waits for the AI response and extracts the generated text.
5. ****Timing End****: Records the end timestamp and calculates the duration (`duration = endTime - startTime`).
6. ****Return****: Returns an object containing the generated email and the `ai\_response\_time\_ms`.

4. Working / API Usage

Endpoint

`POST http://localhost:3000/api/email-template`

Example Input

```
```json
{
 "purpose": "Ask for a meeting next Tuesday",
 "recipient_name": "Alice Smith",
 "tone": "professional"
}
```
```

Example Execution Flow

1. User sends the JSON above.

2. Server receives it, validates "Purpose", "Recipient Name", and "Tone" exist.
3. Server constructs prompt: **"Write a professional email to Alice Smith to ask for a meeting next Tuesday..."**
4. Gemini AI generates the email text.
5. Server calculates time taken (e.g., 500ms).
6. Server responds with:

```
```json
{
 "email_template": "Subject: Meeting Request...\n\nDear Alice...",
 "ai_response_time_ms": 500,
 "purpose": "Ask for a meeting next Tuesday",
 "tone": "professional"
}
```
```